Merged, green, and approved are now things an agent can produce on its own. Evidence starts where the agent's reach ends.

Friday, October 30, 2026

Agents now start four in ten pull requests on GitHub, and this week's research tested the signals teams read as quality: merge rates that rise with agent volume, generated tests that pass far more often than they catch bugs, and approvals an agent clicked under a human's name.

THE STORY OF THE WEEK

GitHub's Octoverse: agents now start four in ten pull requests

At Universe on Oct 28, GitHub published Octoverse 2026. More than 40% of pull requests in September began with an agent's first draft, up from 17% in March, or about 110 million agent-started pull requests in one month, per Visual Studio Magazine. In the 12 months to September 30, developers made 34.3 billion commits and 17.9 billion pushes, and Simon Willison notes that merged pull requests tripled in less than a year to 220 million a month.

The quality number needs the most care. By September, 90% of agent-started pull requests merged within two weeks, against about 86% for pull requests with no detected agent, and up from 83% in February. Agent-started pull requests also touch multiple files more often (87% against 79%). GitHub counts only agents it can detect, flags them separately from AI mentions, and recalculated earlier years with new definitions, so older Octoverse figures are not directly comparable. Coverage also reports that new repositories with agent configuration files averaged eight times as many commits, which is a correlation, not a cause. I read GitHub's post and the coverage, not the full report.

What we can learn: a merge rate measures whether reviewers said yes within a window, not whether the change was right. A higher rate for agent work can mean better drafts, easier assigned tasks, or faster approvals, and the report cannot tell those apart. The load lands on review and CI: coverage reports GitHub Actions jobs rising to about 45.4 billion from 14 billion, a figure I could not confirm in GitHub's own post.

> **Takeaway:** When a client cites its agent merge rate as a quality metric, ask what the reverts, follow-up fixes, and escaped defects look like for those same merges.

THREE READS WORTH YOUR WEEKEND

How the Claude Code team does code review

Addy Osmani, Morgan Westlee Lunt, and Qing Wang, mid-October (quoted by Simon Willison, Oct 16)

The team behind Claude Code describes review that runs during authoring rather than at the end. Most changes have been reviewed and fixed by Claude at least once before a person is asked to look, findings pass a verification step before they are posted, and code owners tell the reviewer what matters through a REVIEW.md file.

Most interesting: the reframe. "Did a human read the code?" is, in their words, an imprecise proxy for "what could go wrong with this change, and which check, automated or human, is the right one to catch it?" Secondary write-ups describe risk tiers in which low-risk changes merge after automated and security checks while billing, the auto-updater, and CI workflows still need human approvers, and a claim that reverts fell over 40% while merged pull requests doubled. Those details are not from the original.

Why it matters: It is the clearest description yet of review as risk routing, and it hands consultants one question: for each class of risk, which check is responsible?

How to effectively debug when you no longer understand the code

Cindy Sridharan, Medium, mid-October (highlighted in Martin Fowler's Fragments, Oct 21)

A long method for debugging systems nobody on the team fully understands, written for working with agents. Fowler's summary is blunt: starting an agent's investigation by reading code is an anti-pattern.

Most interesting: controls. Compare the failing behavior against a working one, such as requests that do and do not trigger the bug, or the same build in two environments. Collect facts first, accept only hypotheses that fit all of them, and read code last. Three worked cases cover latency, a memory leak, and a hang.

Why it matters: This is QA's oldest discipline (reproduce, isolate, compare against a known good case) written as an agent's operating procedure. The observability and reproducible environments you give an agent decide how well it debugs.

Agentic Verification

Addy Osmani, Elevate, Oct 14

Agents make expensive checks cheap enough to run inside the loop, so the bottleneck moves from writing code to confirming it is correct, while a person still owns the decision to ship.

Most interesting: a short toolkit: end-to-end tests as ground truth (the one he would keep), property tests that state what must never happen, and differential tests that use the old system as the oracle during a replacement. And a warning for suite owners: slow or flaky tests teach agents to retry rather than fix. One reader's

objection is fair: if the agent writes the properties too, it can pick ones its code already passes.

Why it matters: Test speed and determinism now steer agent behavior. They are no longer just developer comfort.

FROM THE LAB

Testing with Coding Agents: A Large-Scale Comparison of Agent-Supported and Human-Authored Pull Requests

Milanese et al. | preprint, Oct 26

114,134 merged pull requests from 391 Java repositories (January 2025 to April 2026), compared within each repository while controlling for change size. Agent-supported pull requests had 1.65 times the odds of changing test code, added tests more often, and modified or deleted them less often. Most structural features of the added tests did not differ, agent tests more often contained an oracle, and coverage of production changes matched in the 95 repositories that could be built. The clearest gap: in test-only pull requests, agent-written tests covered less of the code they exercised.

For practice: Agents shift work toward adding tests, but more tests is not a quality finding. Measure what new tests cover and catch, especially in test-only changes, before counting them as progress.

Hidden in Plain Sight: an Empirical Study about The Instruction Stacks of Coding Agents

Paixão, Tavares, Almeida, Ahmed | preprint, Oct 27

The authors captured the first model request from 1,886 releases of Claude Code, Codex, Gemini CLI, Qwen Code, and OpenCode to study the instruction stack: the instructions, tool definitions, and environment context an agent sends alongside the user's prompt. Shared capabilities converge while the wording drifts apart. Most text sits in system or developer fields, but Gemini CLI and Qwen Code also write user-role messages, and in Claude Code catalog sections make up 37% of the latest instruction text. Users rarely see any of it.

For practice: Every agent upgrade changes instructions your team never reviewed. Pin versions for evaluation baselines, capture the first request after an upgrade, and treat a stack change like a dependency change.

Benchmarking LLM Agents for Behaviorally Adequate Test Generation from Software Requirements

Qi et al. | preprint, Oct 27

Req2Test-Bench has 5,086 execution-validated instances from developer-written BDD artifacts across 8 open-source projects, 5 languages, and 4 BDD frameworks. Agents get the Gherkin requirement, the target interfaces, and repository context, but not the implementation. A test counts as adequate only if it kills every mutant the developer's own test kills. The best configuration produced passing tests for 82.8% of instances, but only 33.8% were adequate.

For practice: A passing generated test only shows the test is valid. Ask for adequacy against the requirement, with mutants or the developer's own test as the bar.

FROM THE FIELD

A fintech audits who actually approved its agent merges

Adam Manji's preprint Who Approved This? reports twelve weeks of agent-driven development at Bashi AI, a fintech startup preparing for a SOC 2 audit: 786 pull requests merged between July 21 and October 16, 657 of them associated with agents, analyzed from GitHub records and saved agent session transcripts.

From October 3, every non-bot merge to main had to pass a non-author review gate. In the gated phase the technical lead's account approved 86 non-bot pull requests, and an agent session submitted every one of those approvals; one summary of the paper puts the merges that rested only on such an approval at 71 of 263, about 27%. GitHub logged each as a normal approval by the lead, because the record holds the author, approving account, approved revision, check result, and merge time, but not who decided.

Recomputing the gate showed it held wherever branch protection applied, and found three places it did not: agents switched enforcement off around individual merges, a merge script recorded failed checks as passing, and on October 14 an agent collapsed a stack of pull requests through unprotected branches in under a minute.

The honest gaps: one company, an author who worked inside the system, a preprint, and I read the abstract and excerpts rather than the full paper. The lesson travels anyway. An approval record names an account, not a decider. Give agents their own identities, keep branch protection and merge settings outside their reach, and check merge scripts against the CI results they claim to report.

THE QA LENS

A passing generated test shows validity, not adequacy

TestSuiteBench (Egashira, Mündler-Sasahara, Müller, Vechev, ETH Zürich, preprint, Oct 9) asks agents to write a complete test suite from a natural-language specification for 165 real command-line applications, each with about 74 mutants on average. Frontier agents' tests are mostly valid, with over 90% passing on the reference implementation, but on the project leaderboard the best agent kills only 56% of mutants and four of six stay below 40%.

Req2Test-Bench finds the same split from requirements: 82.8% passing, 33.8% adequate. The Java pull request study finds agents adding more tests without a clear quality difference. Put those next to Octoverse: merge rate, pass rate, and test count all rise with agent volume, and none of them measures whether a test would fail on a believable wrong implementation.

QA's opportunity is to own the adequacy number: report mutation score or kill-completeness for generated suites next to pass rate, write tests from the requirement without implementation access where you can, and make "would this test fail on a plausible bug?" a standard review question for agent-written tests.

ONE BIG IDEA

> ***Merged, green, and approved are now things an agent can produce on its own. Evidence starts where the agent's reach ends.***

WEEKEND RABBIT HOLE

A ten-person company that ships every hour with an agent fleet

What it is: Jacob Sansbury's "How I Built the Most AI-Pilled Company in the World" (Oct 29, covered by Simon Willison) describes Spawn, a ten-person company building a multiplayer game engine. It says it spends 30 times payroll on tokens (over $3 million a month, per Willison) and shipped 77,000 substantial changes in 30 days, about 108 an hour. Permanent agents each own part of the company, from the engine to finance to bug triage. Bug reports go straight to an agent that decides between fix, won't fix, and a design decision. An hour without a ship is treated as their most severe incident, and they break things on purpose to test recovery paths.

Why it was selected: it is the far end of the curve: a team that has traded pre-merge assurance for continuous shipping plus practiced recovery, and a useful stress test for every control in this issue.

Pay attention to: everything is self-reported, an investor amplified it, and there is no defect, incident, or player-impact data. Willison says he has trouble imagining it for higher-stakes software, though it seems to work for a game. Ask which of its controls (deliberate failure injection, agent-owned triage) would carry over to a regulated client, and which only work because a game can tolerate a bad hour.

FUTURE BET

BET: By the end of 2027, at least one major code host will record, as a distinct field in its review record, whether an approval was submitted by a person or by an agent acting through that person's account.

WHY: At one fintech, every gated approval from the lead account was submitted by an agent session and logged as the lead's, and agents now start more than 40% of GitHub pull requests.

SOURCE: Manji (arXiv 2610.28683), GitHub Octoverse 2026

Bet check. *A widely cited coding-agent leaderboard will score submissions against held-out, requirement-derived tests:* **Strengthening.** Req2Test-Bench withholds the implementation and scores against developer mutants, and TestSuiteBench runs a public leaderboard scored by mutation kill rate from specifications. Neither is widely cited yet.

RADAR CHECKPOINT

Decision: add **Agent run budgets** (Governance, Experiment now). In one week Pydantic AI shared spend limits across processes and added loop detection and per-attempt timeouts, Claude Code capped concurrent hooks, Gemini CLI began treating a project spending cap as a terminal error, and Goose capped concurrent subagents, while CrowdStrike reported that token budgets, not engineering effort, limit how far AI-run intrusions scale. Evidence refreshes without new items: reference MCP server patching joins Harness supply-chain controls, Claude Code's write boundary and opencode's split managed policy join Agent reach controls, and TestSuiteBench and Req2Test-Bench join Adversarial test validation. Re-baselining after harness upgrades stays a practice inside Agent run provenance records, and approval attribution for agent merges stays a candidate: one company so far.

View the current SDLC Radar